

Software Requirements Specification for Exam Center Assignment System

Version: 1.0

Prepared by: Group – 01

Revision History

Version	Date	Description
----------------	-------------	--------------------

1.0	—	Initial SRS Document
------------	----------	-----------------------------

Table of Contents

1. Introduction

1.1 Purpose

1.2 Document Conventions

1.3 Intended Audience and Reading Suggestions

1.4 Project Scope

1.5 References

2. Overall Description

2.1 Product Perspective

2.2 Product Functions

2.3 Operating Environment

2.4 Design and Implementation Constraints

2.5 User Documentation

2.6 Assumptions and Dependencies

2.7 System Context Diagram

3. System Features

4. External Interface Requirements

4.1 User Interfaces

4.2 Hardware Interfaces

4.3 Software Interfaces

4.4 Communication Interfaces

5. Other Non-Functional Requirements

5.1 Performance Requirements

5.2 Safety Requirements

5.3 Software Quality Attributes

Other Requirements

Appendix A: Glossary

1. Introduction

The Exam Center Assignment System is a software application developed to automate the process of allocating candidates to examination centers based on the type of examination they are appearing for. The system processes candidate information provided through multiple input text files, validates each entry, and assigns candidates to predefined exam centers while strictly adhering to capacity constraints.

The system is implemented as a standalone, console-based application using the C++ programming language. It leverages object-oriented design principles, multi-threading using POSIX threads, and synchronized access to shared resources to ensure efficiency, accuracy, and reliability. All interactions with the system are performed through file-based input and output operations.

This document provides a detailed description of the software requirements and serves as a foundational reference for system design, development, testing, and maintenance.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to clearly and comprehensively describe the functional and non-functional requirements of the Exam Center Assignment System.

This document aims to:

- Define the expected behaviour of the system
- Specify system constraints and operating conditions
- Serve as a communication medium between stakeholders, developers, and testers
- Act as a baseline reference for High-Level Design (HLD) and Low-Level Design (LLD)
- Assist in verification and validation activities during testing

The SRS ensures that all system requirements are well understood before implementation begins and that the final system meets the intended objectives.

1.2 Document Conventions

The following document conventions are used throughout this SRS to maintain consistency and readability:

- **Font Style:** Times New Roman
- **Main Headings:** 18-point font, bold
- **Sub-headings:** 14-point font, bold
- **Body Text:** 12-point font
- **Alignment:** Left-aligned text
- **Spacing:** Standard MS Word spacing

Additional conventions:

- Requirement identifiers are written in uppercase (e.g., EC01, EC-NR01)
- Mandatory requirements are explicitly labeled as *Mandatory*

- Optional requirements are explicitly labeled as *Optional*
- Technical terms and acronyms are expanded on first use and listed in the glossary

1.3 Intended Audience and Reading Suggestions

This document is intended for the following audience:

- **Project Stakeholders:** To understand system capabilities and limitations
- **Software Developers:** To implement system functionality as specified
- **System Architects and Designers:** To derive HLD and LLD from requirements
- **Test Engineers:** To design unit, integration, and system test cases
- **Academic Evaluators and Reviewers:** To assess completeness and correctness of requirements

Reading Suggestions:

- Readers new to the project should begin with **Section 1 (Introduction)** and **Section 2 (Overall Description)** to gain a conceptual understanding of the system.
- Developers should focus on **Section 3 (System Features)** for detailed functional requirements.
- Testers should review **functional and non-functional requirements** to derive test scenarios.
- Reviewers may refer to the glossary for technical terminology.

1.4 Project Scope

The scope of the Exam Center Assignment System includes the following functionalities:

- Reading candidate information from multiple input text files provided as command-line arguments
- Validating candidate entries and identifying invalid records
- Assigning candidates to exam centers based on examination type
- Managing fixed exam center capacities and handling overflow cases
- Generating output files for:
 - Exam center-wise candidate lists
 - Unassigned candidates
 - Invalid candidate entries
- Displaying and optionally storing hall tickets for candidates
- Supporting concurrent processing of input files using multi-threading

Out of Scope:

- Online or network-based interfaces
- Database integration

- User authentication or role-based access
- Real-time exam scheduling or seat selection

1.5 References

The following references were used during requirement analysis and system design:

- IEEE Software Engineering Standards for SRS Documentation
- POSIX Threads (pthread) Documentation
- C++ Standard Library Documentation
- GNU Compiler Collection (GCC) Documentation

2. Overall Description

This section provides an overview of the system from a high-level perspective. It describes how the system fits into its operational environment, the major functionalities it offers, and the constraints under which it operates.

2.1 Product Perspective

The Exam Center Assignment System is a **standalone application** that operates independently of external systems. It is designed as a **modular and layered system**, consisting of:

- Input Processing Layer
- Validation Layer
- Exam Center Assignment Logic
- Output Generation Layer

The system architecture follows object-oriented principles, with a base Candidate class and specialized derived classes for different exam types. Shared exam center resources are protected using mutex locks to ensure thread-safe operations.

2.2 Product Functions

The major functions of the system are as follows:

- Accept multiple candidate data files as input
- Parse and store candidate records
- Validate candidate information and detect errors
- Assign candidates to appropriate exam centers
- Enforce exam center capacity constraints
- Generate exam center-specific candidate lists
- Record unassigned and invalid candidates

- Display and store hall tickets upon request

Each function is executed in accordance with the system's design constraints and performance requirements.

2.3 Operating Environment

The system operates under the following environment:

- **Operating Systems:** Linux and Windows
- **Programming Language:** C++
- **Compiler:** GNU C++ Compiler (g++)
- **Execution Mode:** Command Line Interface (CLI)
- **Threading Library:** POSIX pthread
- **Build Tool:** Makefile

2.4 Design and Implementation Constraints

The system is subject to the following constraints:

- Implementation must be done using C++
- Object-Oriented Programming concepts must be followed
- POSIX threads must be used for multi-threading
- One mutex must be used per exam center
- Input and output must be file-based
- Fixed number of exam centers (four)
- No database or network dependency

2.5 User Documentation

User documentation provided with the system includes:

- README file describing application usage
- Instructions for compiling and executing the application
- Input file format specification
- Sample input and output files
- Error handling and troubleshooting guide

2.6 Assumptions and Dependencies

Assumptions:

- Input files are correctly formatted
- Required software tools are installed

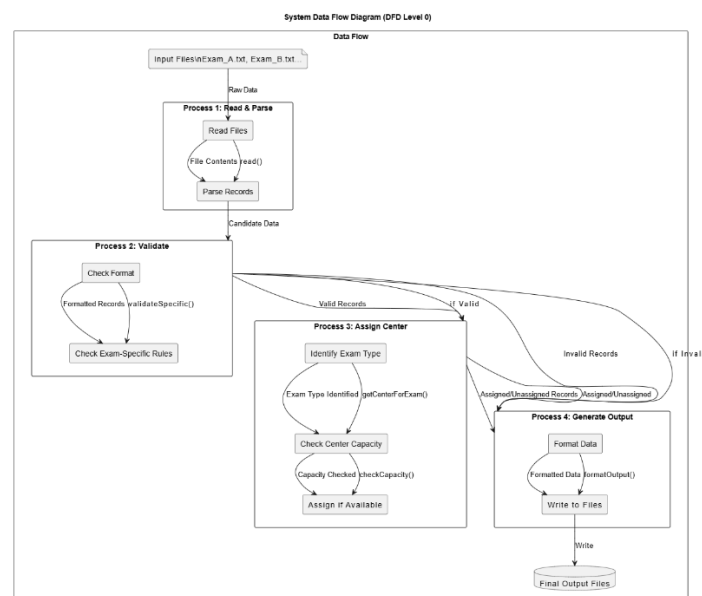
- System is executed by an authorized administrator
- System clock and OS environment are correctly configured

Dependencies:

- Availability of GCC compiler
- POSIX thread support in the operating system
- File system permissions for read/write operations
- Proper configuration of Makefile and directory structure

2.7 System Context Diagram

This diagram provides a high-level view of the Exam Center Assignment System, showing external entities, inputs, and outputs. Detailed design diagrams such as class and sequence diagrams are included in the HLD and LLD documents.



3. System Features

This section describes the major functional capabilities of the system. The system is designed as a **standalone, multithreaded application** using **POSIX threads**, focused on parallel processing and shared-memory coordination.

3.1 System Feature 1: Concurrent File Processing

Feature Description

The system supports concurrent processing of multiple input files using POSIX threads. Each input

file is processed independently, allowing faster execution and improved performance when handling large candidate datasets.

Functional Characteristics

- Each input file shall be processed by a separate POSIX thread.
- Threads shall parse, validate, and extract candidate data independently.
- All threads shall execute within the same process space.
- Thread creation and termination shall be handled dynamically based on the number of input files.
- Thread execution shall not block other threads during file processing.

Implementation Notes

- POSIX `pthread_create()` shall be used to spawn threads.
- Each thread shall receive the filename as its input parameter.
- Threads shall run concurrently and complete independently.

3.2 System Feature 2: Shared Resource Synchronization

Feature Description

The system manages shared resources such as exam center capacities and candidate lists. Since multiple threads may attempt to update shared data simultaneously, synchronization is required to ensure data consistency and correctness.

Functional Characteristics

- Shared resources shall be protected using mutex locks.
- One mutex shall be assigned per exam center.
- Only one thread shall be allowed to modify an exam center's data at a time.
- Threads attempting to access a locked resource shall wait until the lock is released.
- Deadlocks shall be avoided through disciplined lock acquisition and release.

Implementation Notes

- POSIX mutexes (`pthread_mutex_t`) shall be used.
- Mutex locking shall occur only during critical sections.
- Mutexes shall be initialized during system startup and destroyed at shutdown.

3.3 System Feature 3: Candidate Assignment Logic

Feature Description

The system assigns candidates to exam centers based on exam type while respecting capacity constraints. This logic is executed concurrently by multiple threads but coordinated through synchronized access.

Functional Characteristics

- Each validated candidate shall be matched to a predefined exam center.
- Exam center capacity shall be checked before assignment.
- If capacity is exceeded, the candidate shall be marked as unassigned.
- Assignment operations shall be thread-safe.
- Assignment results shall be recorded consistently across all threads.

4. External Interface Requirements

This section defines all external interaction points of the system. Since the system is **non-networked**, interfaces are limited to **users, hardware, software components, and thread-based internal communication**.

4.1 User Interfaces

The system provides a **command-line interface (CLI)** intended for administrators and technical users.

- Users shall execute the application via terminal.
- Input file names shall be provided as command-line arguments.
- The system shall display processing status, errors, and summary information on the console.
- Error messages shall be clear and descriptive.
- No graphical interface is required or supported.

4.2 Hardware Interfaces

The system does not interact directly with specialized hardware devices.

- The system shall run on standard desktop or server hardware.
- Hardware resources such as CPU, memory, and disk shall be managed by the operating system.
- No hardware-specific drivers or configurations are required.
- Performance shall scale with available CPU cores due to multithreading.

4.3 Software Interfaces

The system interfaces with operating system services and libraries required for execution.

- Operating System:
 - Linux or POSIX-compliant systems
- Libraries and APIs:
 - POSIX Threads (pthread)

- Standard C++ libraries
 - STL containers for data storage
- System utilities:
 - File system for reading input and writing output files
- Build tools:
 - Makefile-based compilation
 - GCC / g++ compiler

4.4 Communication Interfaces (POSIX Multithreading Based)

Important:

This system does **not** use network communication. All coordination and communication occur **within a single process** using POSIX multithreading constructs.

Thread Communication Model

- Threads communicate implicitly through **shared memory**.
- Shared data structures shall reside in the same process address space.
- No inter-process or socket-based communication shall be used.

Synchronization Mechanisms

- Mutexes shall be used to control access to shared resources.
- Condition variables may be used where threads need to wait for specific conditions.
- Thread joins shall be used to ensure all threads complete before system termination.

Execution Flow Coordination

- The main thread shall:
 - Initialize shared resources
 - Create worker threads
 - Wait for all worker threads to complete
- Worker threads shall:
 - Process assigned input files
 - Safely update shared data
 - Terminate after completion

Data Exchange Guarantees

- Data consistency shall be ensured through strict synchronization.
- No race conditions shall be permitted.
- Partial updates to shared resources shall never be visible to other threads.

Failure Handling

- Thread failures shall be isolated and logged.
- A failure in one thread shall not crash the entire application.
- Resources shall be released gracefully during abnormal termination.

5. Other Non-Functional Requirements

This section defines the non-functional characteristics of the system. These requirements describe **how the system behaves**, focusing on performance, safety, and quality attributes rather than specific functionalities.

5.1 Performance Requirements

The system is designed to efficiently process multiple input files concurrently using POSIX multithreading.

- The system shall process multiple input files in parallel, with each file handled by a separate thread.
- The application shall demonstrate improved execution time when additional CPU cores are available.
- The system shall support a minimum of 10 concurrently running threads without performance degradation.
- Thread creation, execution, and termination shall incur minimal overhead.
- Average file processing latency shall not exceed acceptable limits under normal load.
- Mutex locking duration shall be minimized to reduce thread contention.
- The system shall complete execution within reasonable time limits even for large input datasets.
- Resource utilization shall remain within system limits:
 - CPU usage proportional to workload
 - Memory usage controlled via efficient data structures

5.2 Safety Requirements

Safety requirements ensure the system executes correctly without causing data corruption, system crashes, or unstable behavior.

- The system shall prevent race conditions through proper synchronization mechanisms.
- All shared resources shall be accessed only through protected critical sections.
- Deadlocks shall be avoided through consistent mutex usage.
- Invalid input data shall be handled gracefully without terminating the application.
- Memory allocated dynamically shall be released appropriately to avoid memory leaks.
- A failure in one thread shall not affect the execution of other threads.

- The system shall terminate gracefully even in the presence of unexpected errors.
- File access errors shall be detected and reported clearly to the user.

5.3 Software Quality Attributes

The system shall meet the following quality attributes to ensure long-term reliability and maintainability.

Reliability

- The system shall produce consistent and correct results across multiple executions.
- Exam center assignments shall remain accurate and deterministic.
- Thread execution order shall not affect final output correctness.

Maintainability

- The system shall be modular, with clearly defined responsibilities for each component.
- Code shall follow standard C++ coding conventions.
- Internal documentation and comments shall explain complex logic.
- Changes to exam centers or rules shall require minimal code modification.

Portability

- The system shall compile and run on POSIX-compliant operating systems.
- No platform-specific dependencies beyond POSIX threads shall be introduced.

Scalability

- The system shall scale with additional CPU cores.
- The number of input files and candidates processed shall be limited only by system resources.

Testability

- Individual modules shall be testable independently.
- Multithreaded behavior shall be verifiable using controlled test inputs.
- Memory usage shall be validated using memory analysis tools.

Usability

- The system shall be easy to execute using simple command-line instructions.
- Output files shall be human-readable and clearly structured.
- Error messages shall assist users in correcting input issues.

6. Other Requirements

This section captures additional requirements related to development, documentation, and testing.

- The system shall be implemented using C++.
- POSIX threads shall be used for multithreading.
- Standard Template Library (STL) containers shall be used where appropriate.
- A Makefile shall be provided for building the application.
- The project shall follow a multi-file and multi-directory structure.
- Memory leak detection shall be performed using analysis tools.
- Unit testing shall be conducted for critical modules.
- High-Level Design (HLD) and Low-Level Design (LLD) documents shall be provided.
- All project documentation shall be submitted in PDF or DOCX format.

Appendix A: Glossary

Term	Description
POSIX	Portable Operating System Interface
Thread	Smallest unit of execution within a process
Mutex	Synchronization mechanism to protect shared resources
Critical Section	Code region where shared data is accessed
Deadlock	Condition where threads wait indefinitely for resources
Race Condition	Error caused by unsynchronized access to shared data
SRS	Software Requirements Specification
HLD	High Level Design
LLD	Low Level Design
STL	Standard Template Library
CLI	Command Line Interface
CPU	Central Processing Unit
RAM	Random Access Memory
Makefile	Script used to automate program compilation